Bakery Algorithm: Analisi Approfondita

Introduzione

Il **Bakery Algorithm** è un algoritmo di sincronizzazione utilizzato in sistemi concorrenti per implementare l'accesso mutuamente esclusivo a risorse condivise. Il nome deriva dalla sua somiglianza con il sistema di gestione delle code in una panetteria: i clienti ricevono un ticket numerato e vengono serviti in ordine.

Backery

Parto dal concept di un negozio con un ticket e tante persone che aspettano il proprio turno.

- Non servono read e write come operazioni atomiche, posso leggere mentre altri scrivono.

```
// dichiarazione delle variabili globali comuni
bool sceglie[N] = {false}; // N costante numero di processi
int numero[N] = {0};

int i; // indice del thread in esecuzione
// ...
while (numero[i] == 0) {
    //prendo il numero
    sceglie[i] = true;
    numero[i] = 1 + max(numero[0], numero[1], ..., numero[N - 1]);//il più grande tra tutti i
numeri in possesso
    sceglie[i] = false; //ho preso il numero
    for j in 1 .. N except i {
        while (sceglie[j] == true); //aspetto la scelta di altri
        while (numero[j] != 0 && (numero[j],j) < (numero[i],i)); //controllo di avere il
numero più basso, dopo i vari zeri
}

    // <sezione critica>
    numero[i] = 0;
    // <sezione non critica>
}
```

Concetto Fondamentale

Il Bakery Algorithm parte dal concetto di un negozio con un ticket system, dove múltiples persone aspettano il proprio turno per essere servite. Nel contesto della programmazione concorrente:

- **N processi** competono per accedere a una **sezione critica** condivisa
- Ogni processo riceve un **numero di ticket** univoco e progressivo
- I processi vengono serviti rigorosamente **in ordine numerico**

- Non è richiesto supporto hardware per operazioni atomiche (read/write non sono atomiche)

Vantaggi Rispetto ad Altre Soluzioni

- **Correttezza garantita**: Anche con operazioni read/write non atomiche, un processo può leggere i dati di altri processi mentre questi scrivono
- **Equità (Fairness)**: L'ordine FIFO (First In First Out) è rigorosamente mantenuto
- **Implementabilità**: Non richiede istruzioni speciali del processore (test-and-set, compare-and-swap)

Strutture Dati Principali

Variabile	Descrizione
sceglie[N]	Array booleano che indica se il processo i sta scegliendo un numero
numero[N]	Array di interi che memorizza il numero di ticket di ogni processo
i	Indice del thread in esecuzione

Table 1: Strutture dati nel Bakery Algorithm

Logica Algoritmica

Fase 1: Scelta del Numero di Ticket

```
sceglie[i] = true;
numero[i] = 1 + max(numero[0], numero[1], ..., numero[N-1]);
sceglie[i] = false;
```

In questa fase:

1. Il processo i segnala che sta scegliendo un numero (`sceglie[i] = true`)
2. Calcola il suo numero di ticket come **uno più grande rispetto a tutti i numeri attualmente assegnati**
3. Completa la scelta (`sceglie[i] = false`)

Il flag `sceglie[i]` è cruciale: protegge altri processi dal leggere un numero incompleto o incoerente durante la fase di aggiornamento.

Fase 2: Attesa e Comparazione

```
for j = 1 to N except i {
while (sceglie[j] == true); // Aspetta che j finisca di scegliere
while (numero[j] != 0 && (numero[j], j) < (numero[i], i));
}
```

Per ogni altro processo j:

1. **Attesa della scelta**: Se j sta ancora scegliendo il suo numero, aspetta
2. **Comparazione lessicografica**: Aspetta fino a che il numero di j non è 0 (libero) OPPURE il numero di j è maggiore del nostro
    - La coppia (numero, indice) viene comparata lessicograficamente
    - Questo assicura un ordine totale e risolve i tie (quando due processi hanno lo stesso numero)

Fase 3: Sezione Critica

numero[i] = 0; // <sezione critica>

Il processo entra nella sezione critica in sicurezza. Solo un processo alla volta può trovarsi qui.

Fase 4: Uscita dalla Sezione Critica

numero[i] = 0; // <sezione non critica>

Il processo rilascia il suo ticket azzerando numero[i], segnalando che è disponibile per altri processi.

Proprietà di Correttezza

Mutua Esclusione (Mutual Exclusion)

Teorema: In qualsiasi istante, al massimo un processo può trovarsi nella sezione critica.

Dimostrazione intuitiva:

- Supponiamo due processi i e j siano entrambi nella sezione critica
- Senza perdita di generalità, i ha un numero più basso (o il numero più basso con indice inferiore)
- Però i non potrebbe aver superato il controllo while per j se j fosse stato ancora nella sezione critica
- Contraddizione → Assurdo

Assenza di Deadlock (Deadlock-Freedom)

Teorema: Se un processo richiede di entrare nella sezione critica, prima o poi riuscirà ad accedervi.

Proprietà: L'ordine totale definito dalle coppie (numero[j], j) garantisce che non vi siano cicli di attesa.

Fairness (Equità)

Teorema: Se il processo i richiede l'accesso, i processi che hanno richiesto accesso prima entreranno nella sezione critica prima di i.

Proprietà: L'ordine FIFO è garantito dal numero di ticket progressivo.

Analisi del Codice Fornito

Il codice nell'immagine implementa correttamente il Bakery Algorithm:

Linea	Operazione	Scopo
`sceglie[i] = true`	Segnala inizio selezione	Protegge il calcolo del numero
`numero[i] = 1 + max(...)`	Calcola nuovo ticket	Assicura numero univoco e ordinato
`sceglie[i] = false`	Completa selezione	Notifica disponibilità numero
`while (sceglie[j] == true)`	Attesa selezione altrui	Evita letture incoerenti
`while (numero[j] != 0 && ...)`	Attesa turno	Aspetta processi con priorità maggiore
`numero[i] = 0`	Rilascio turno	Libera il ticket

Complessità e Prestazioni

Complessità Computazionale

- **Spazio**: O(N) per i due array globali
- **Tempo per processo**: $O(N^2)$ nel caso peggiore (due loop annidati)
- **Contention**: La contesa di accesso degrada linearmente con N

Limitazioni Pratiche

1. **Scalabilità**: Non è efficiente per grandi N (centinaia o migliaia di processi)
2. **Cache coherence**: In architetture multi-core con cache, il continuo accesso ai dati condivisi causa cache invalidation
3. **Overhead di sincronizzazione**: Per piccoli N (<4-8), lock hardware sono più veloci

Confronto con Altre Soluzioni

Proprietà	Bakery	Semaphore	Lock	CAS
Mutua Esclusione	✓	✓	✓	✓
Fairness	✓	✗	✗	✗
Hardware Support	✗	✗	✓	✓
Scalabilità	Bassa	Media	Alta	Alta
Semplicità	Media	Alta	Alta	Bassa

Table 2: Confronto tra algoritmi di sincronizzazione

Casi di Utilizzo

Ambienti Ideali per Bakery Algorithm

1. **Sistemi embedded** con vincoli hardware limitati
2. **Algoritmi didattici** per insegnare sincronizzazione
3. **Situazioni con bassa contesa** (pochi processi)
4. **Sistemi dove la fairness è critica** (schedulazione temps réel)

Quando Evitare

1. **Sistemi moderni multi-core** (prefirire lock OS o CAS atomiche)
2. **Applicazioni con alta contesa** (contention killer)
3. **Quando le prestazioni sono critiche**

Esempio Pratico: Scenario di Panetteria

Immaginiamo 5 clienti che entrano in una panetteria:

T0: Cliente 1 prende ticket n=1, entra nella sezione critica
T1: Cliente 2 arriva, assegna ticket n=2, aspetta
T2: Cliente 3 arriva, assegna ticket n=3, aspetta
T3: Cliente 1 finisce, azzera il suo numero → n=0
T4: Cliente 2 (n=2) entra nella sezione critica
T5: Cliente 4 arriva, assegna ticket n=4, aspetta
... e così via in ordine perfetto FIFO

Conclusioni

Il Bakery Algorithm rappresenta una soluzione elegante e teoricamente significativa al problema della mutua esclusione in ambienti concorrenti. Sebbene non sia optimale per sistemi moderni ad alte prestazioni, rimane un importante:

- **Strumento educativo** per comprendere i principi di sincronizzazione
- **Prova di concetto** che la mutua esclusione può essere implementata senza supporto hardware speciale
- **Ispirazione** per algoritmi più sofisticati e efficienti

La sua proprietà di fairness lo rende particolarmente interessante in contesti dove l'ordine d'accesso è importante, come sistemi in tempo reale e sistemi critici per la sicurezza.

Riferimenti e Approfondimenti

Per comprendere meglio l'algoritmo, considera di studiare:

1. **Lamport, L. (1974)**: "A New Solution of Dijkstra's Concurrent Programming Problem" (articolo originale)
2. **Algoritmi alternativi**: Peterson Algorithm, Lamport's Fast Mutual Exclusion
3. **Architetture hardware**: Memory barriers, Cache coherence protocols
4. **Sistemi operativi moderni**: Come pthread_mutex, futex, spinlock implementano sincronizzazione